



Processing Data for ML

AWS Academy Data Engineering

Introduction

Processing Data for ML

Module objectives

This module prepares you to do the following:

- Distinguish between labels, features, and samples in the context of machine learning (ML).
- Describe each phase of the ML lifecycle and the roles involved in each.
- Cite questions to ask when framing an ML problem.
- Discuss key considerations and tasks during the data collection phase of the ML lifecycle.
- Give examples of preprocessing and feature engineering data transformations.
- Compare the activities and resource considerations for model development and model deployment.
- Categorize AWS ML infrastructure services based on how they might be used in the ML lifecycle.
- Describe features of Amazon SageMaker that assist with each phase of the ML lifecycle.
- Define generative artificial intelligence (AI).
- Explain how Amazon Q Developer works with Jupyter notebooks to optimize and speed up development of your ML application.
- Cite two AWS services that use ML for common use cases.

Module overview

Presentation sections

- ML concepts
- The ML lifecycle
- Framing the ML problem to meet the business goal
- Collecting data
- Applying labels to training data with known targets
- Preprocessing data
- Feature engineering
- Developing a model
- Deploying a model
- ML infrastructure on AWS
- SageMaker
- Working with generative AI
- AI/ML services on AWS

Activity

- Labeling with SageMaker Ground Truth

Demos

- Preparing Data and Training a Model with SageMaker
- Preparing Data and Training a Model with SageMaker Canvas
- Creating a Linear Regression Model Using Amazon Q Developer

Knowledge checks

- Online knowledge check
- Sample exam question

ML concepts

Processing Data for ML

Comparing ML to traditional analytics

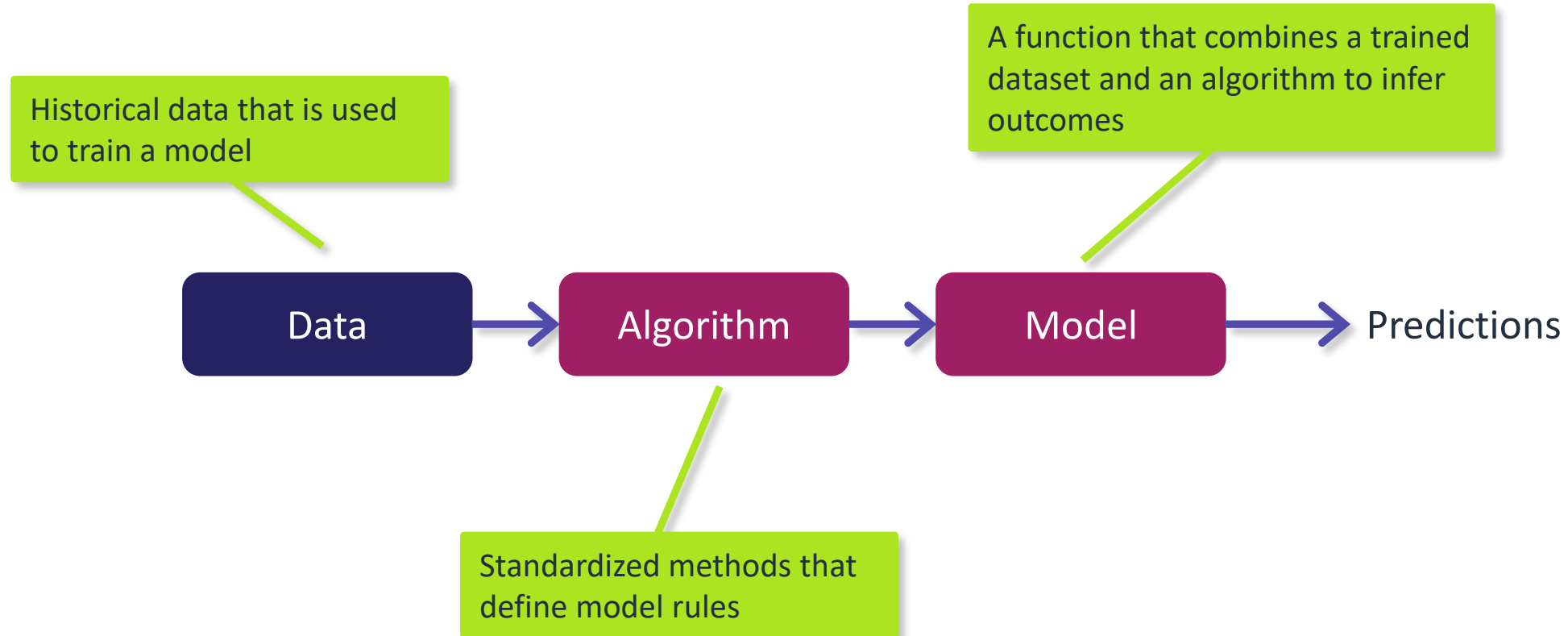
Traditional analytics

- Is the systematic analysis of large datasets (big data) to find patterns and trends to produce actionable insights
- Uses programming logic to answer questions from data
- Is good for structured data with a limited number of variables

Machine learning

- Is a set of mathematical models that are used to make predictions from data at a scale that is difficult or impossible for humans
- Uses examples from large amounts of data to learn about the data and answer questions
- Is good for unstructured data and where the variables are complex

Algorithms are used to train ML models



Three general types of ML models

Supervised

The model is given inputs and related outputs in the training data.

Example: Identify fraudulent transactions using past examples.

Unsupervised

The model finds patterns in the training data without help.

Example: Group users with similar viewing patterns to make recommendations.

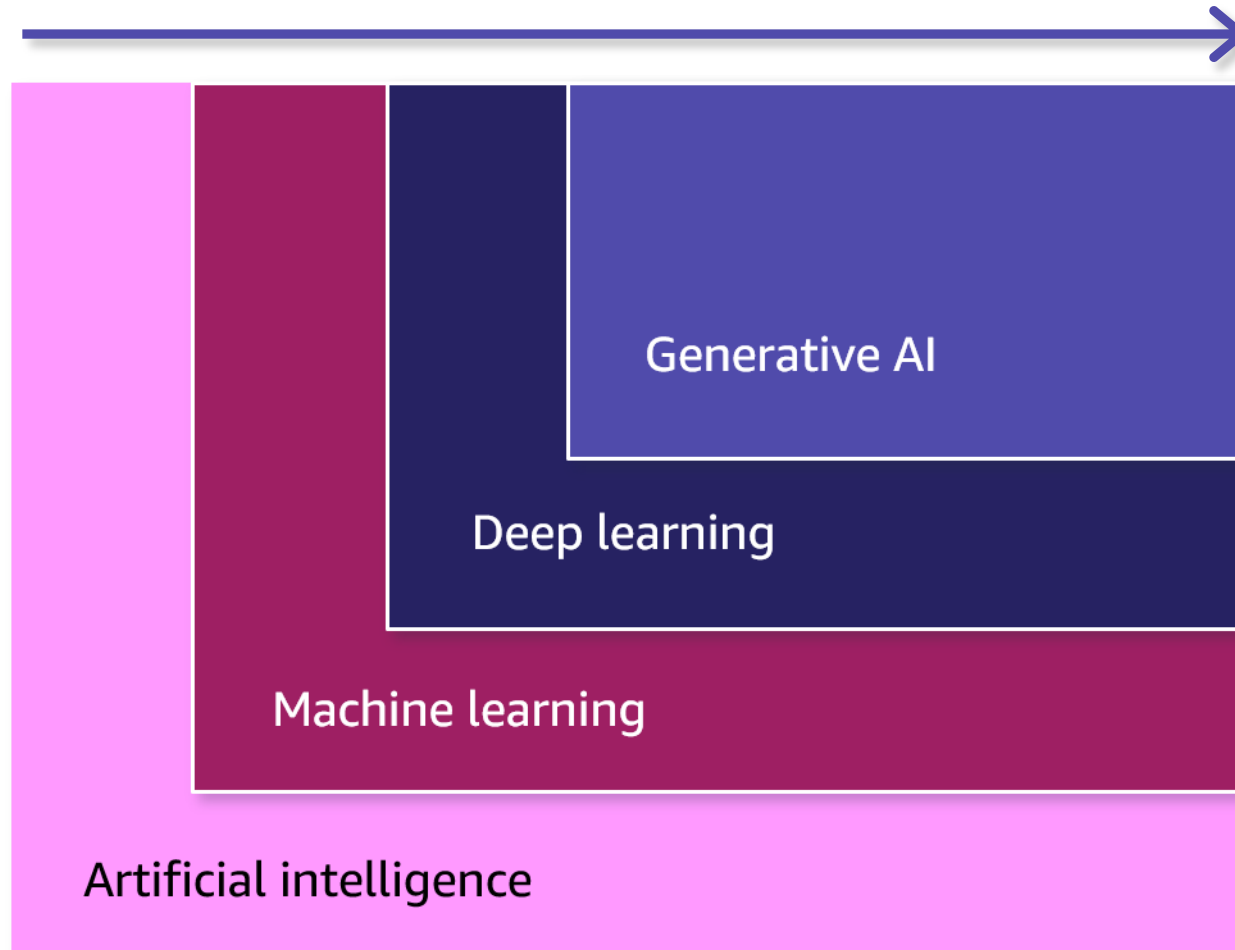
Reinforcement

The model learns from its environment and takes actions to maximize rewards.

Example: Develop self-driving cars.

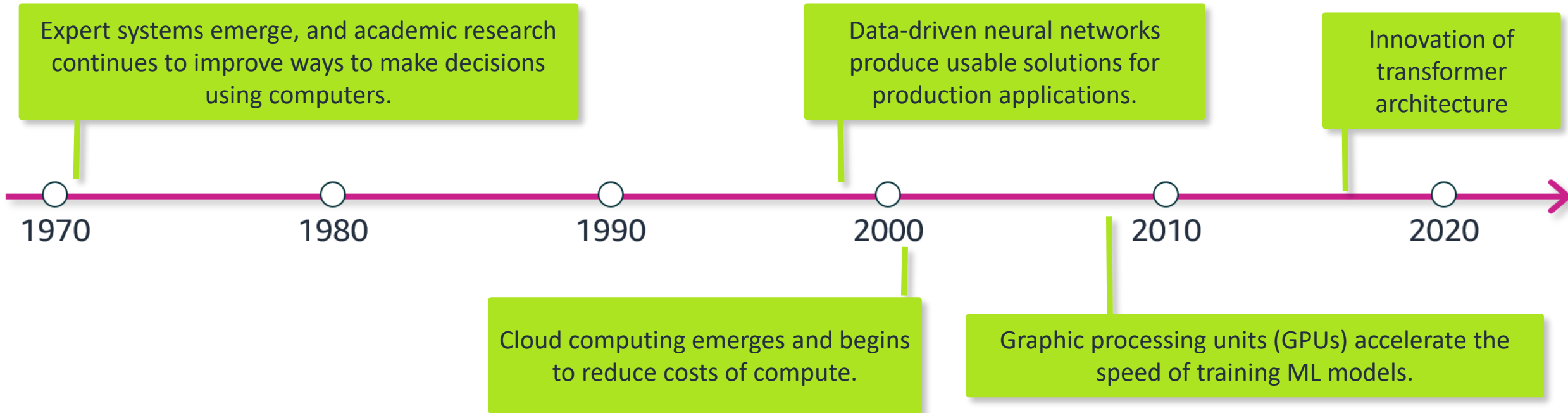
Subcategories of AI

Increasing complexity of problems it can solve →



↑
Increasing compute
power required

The evolution of ML for practical use



ML data concepts

Target (label)
↓

	months_as_customer	age	umbrella_limit	vehicle_claim	Is this fraud?
	328	48	0	52080	
	228	42	5000000	3510	
	134	29	5000000	23100	
Sample	256	41	6000000	50720	
	228	44	6000000	4550	

Features

Key takeaways: ML concepts

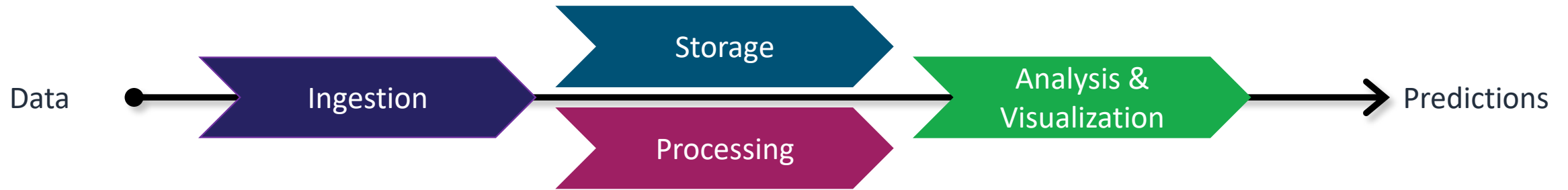


- An ML model is a function that combines a trained dataset and an algorithm to predict outcomes.
- The three general types of machine learning include supervised, unsupervised and reinforcement.
- Deep learning is a subcategory of machine learning that uses neural networks to develop models.
- Generative AI is a subcategory of deep learning that can generate content and is trained on large amounts of data.
- In machine learning, the label or target is what you are trying to predict, and the features are attributes that can be used to predict the target.

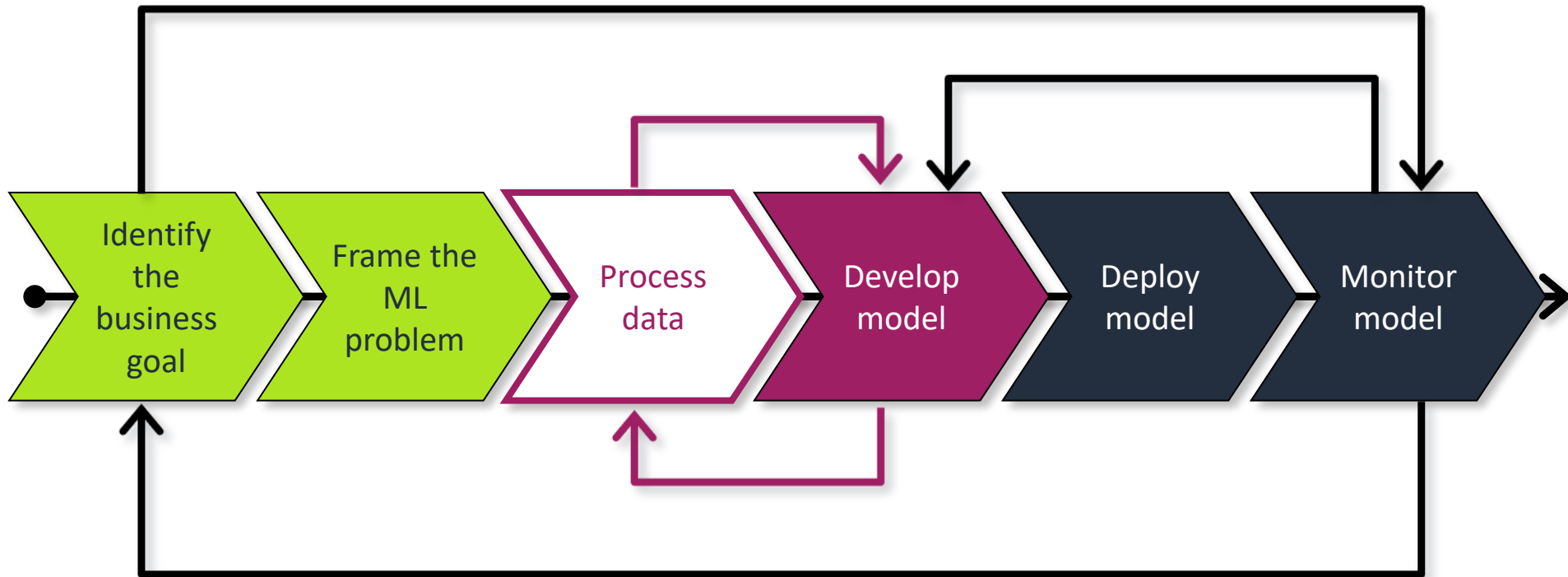
The ML lifecycle

Processing Data for ML

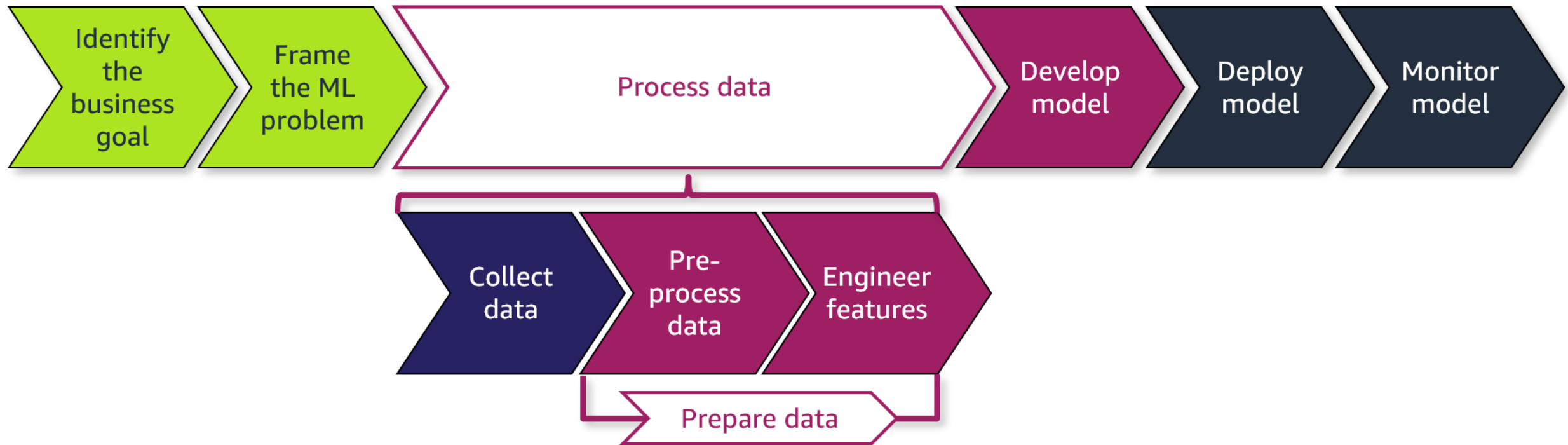
ML processing involves all layers of the data pipeline



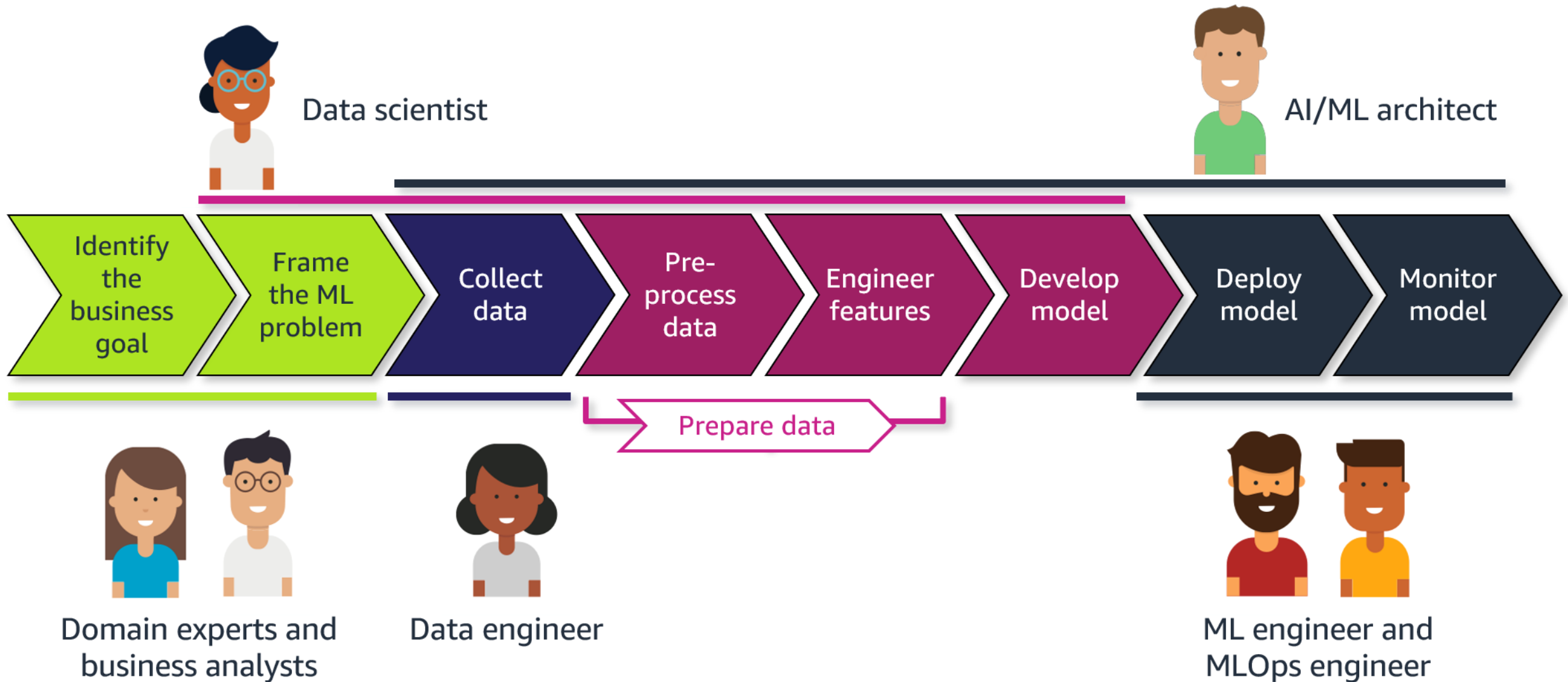
The ML lifecycle defines a set of iterative phases



The process data phase includes collecting and preparing data



Common roles in the ML lifecycle



Key takeaways: The ML lifecycle



- The ML lifecycle starts with defining a business problem and framing it as an ML problem.
- The next step is to collect data and prepare it to use in an ML model.
- Model development takes the prepared data and then hands it off to be deployed to production when the model is ready.
- The final phase in the lifecycle is to monitor the model in production.

Framing the ML problem to meet the business goal

Processing Data for ML

Working backwards from the business problem to be solved



- What is the business problem?
- What pain is it causing?
- Why does this problem need to be resolved?
- What will happen if you don't solve this problem?
- How will you measure success?

Example: Problem statement and business goal

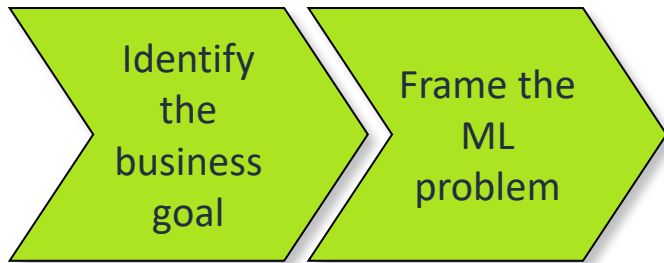
Problem statement: Recently, AnyCompany (a car insurance provider) has seen an increase in global insurance fraud, which is costing them millions of dollars per year in financial losses, administrative overhead, and investigation activity.



Identify
the
business
goal

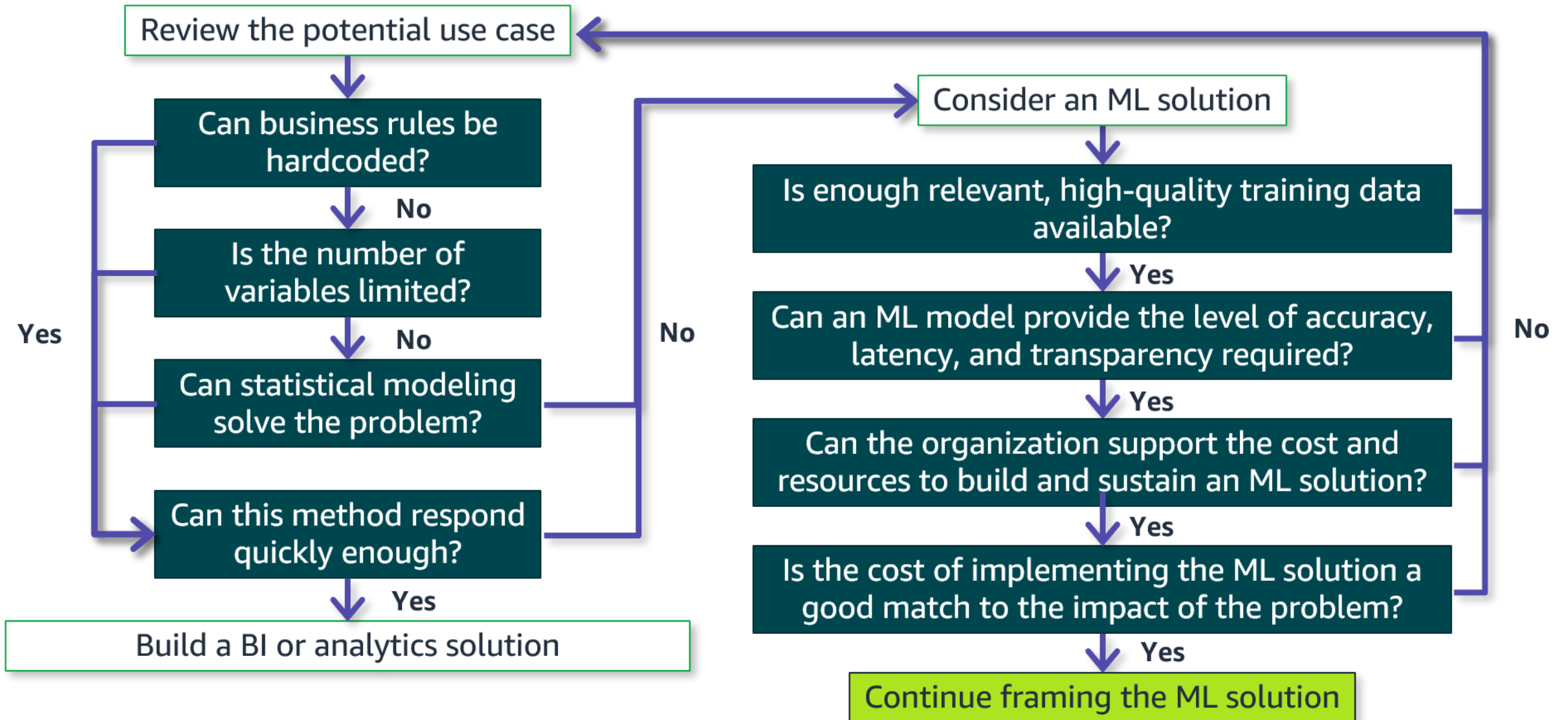
Business goal: Improve our process to predict fraud for new claims by the end of the calendar year.

Key steps in framing the ML problem



- Determine what will be observed (**feature**) and what will be predicted (**label** or **target**).
- Establish observable, quantifiable metrics.
- Create a strategy for data sourcing.
- Evaluate whether ML is the correct approach.
- Start with a simple model and iterate.

Determining if ML is the best approach



Example: ML problem framing

- A data scientist for AnyCompany worked with a domain expert for car insurance claims who identified a set of relational database tables with information about the prior year's claims. The domain expert also has the information necessary to identify which claims turned out to be fraudulent.
- They have approximately 1,000 available claims records from the previous 12-month period.
- The availability of labeled data makes this a good candidate for supervised learning.
- The target (Is this fraud?) is a binary classification problem (the answer is one of two choices—yes or no). The data scientist plans to use an open-source binary classification algorithm.

Key takeaways: Framing the ML problem to meet the business goal

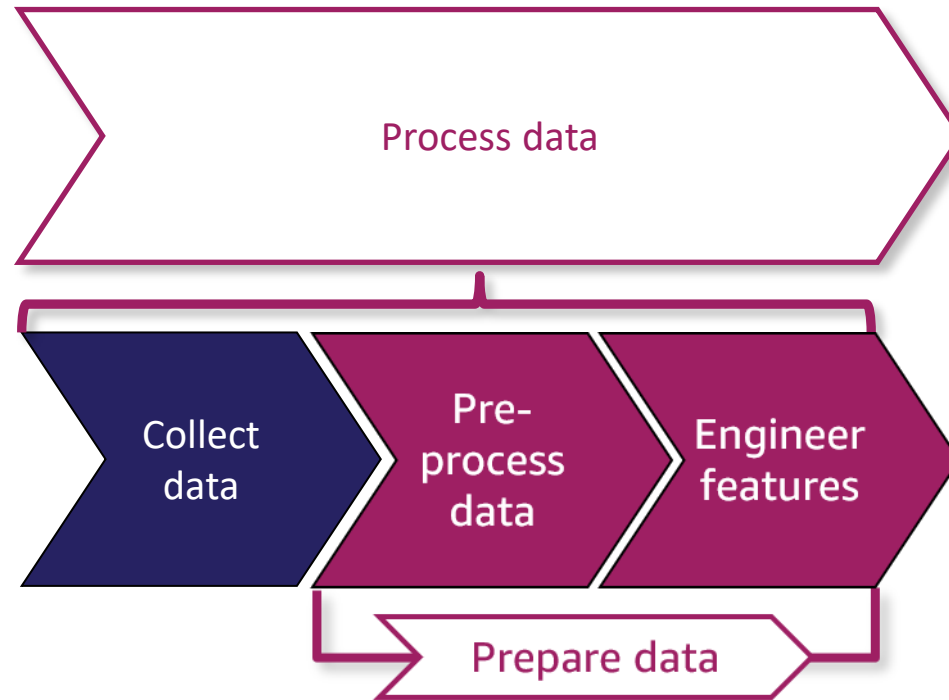


- State the business goal in the form of a problem statement.
- Data scientists work with domain experts to determine an appropriate ML approach.
- The business problem drives whether ML is needed or if a simpler solution can meet the need.

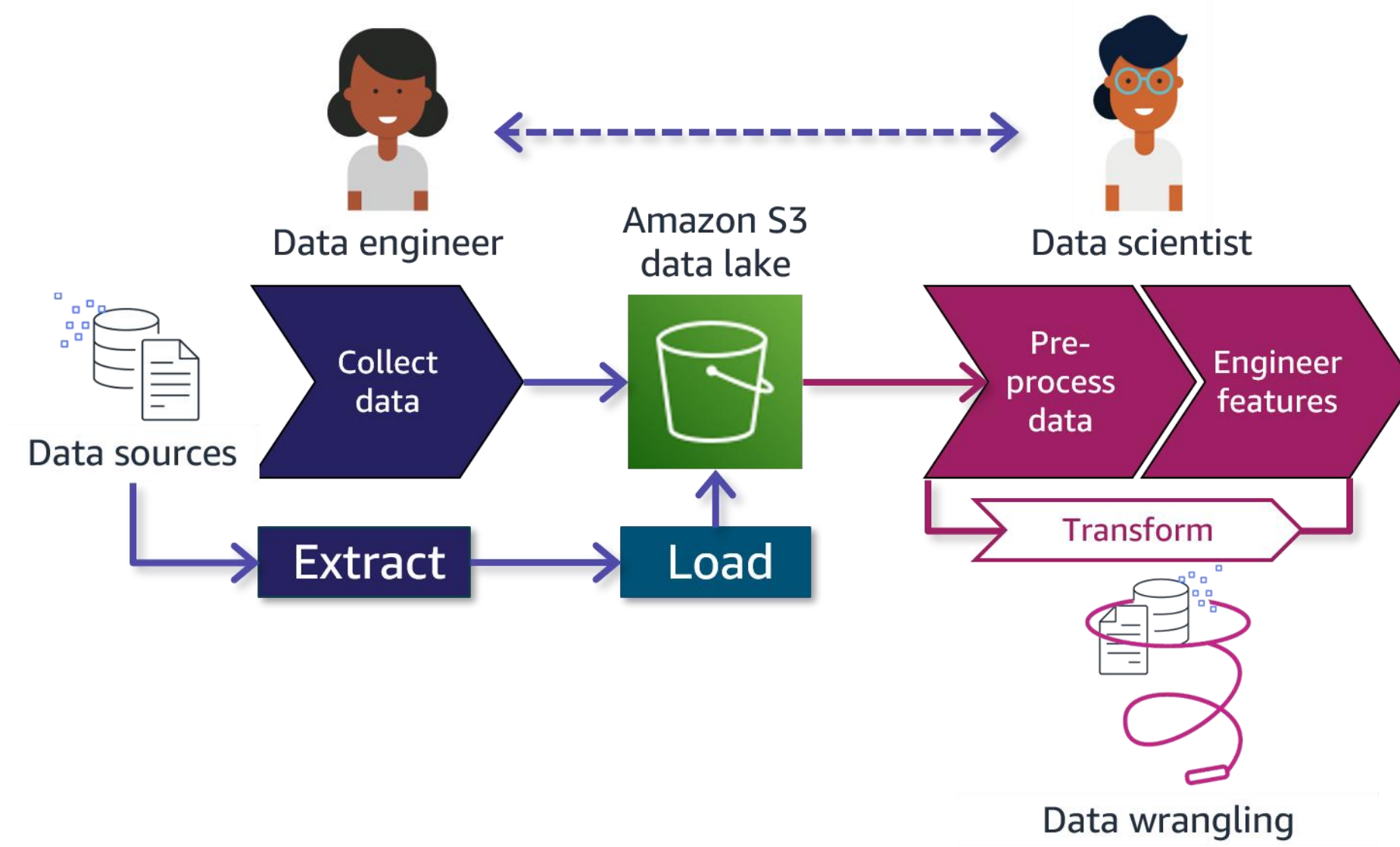
Collecting data

Processing Data for ML

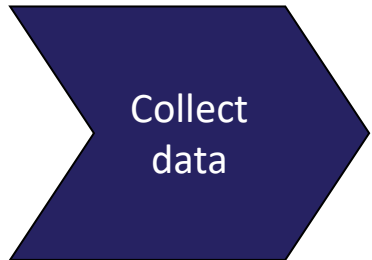
Collecting data is the first step of processing data for ML



The data engineer builds the ingestion pipeline



Key steps in collecting data to be used in ML



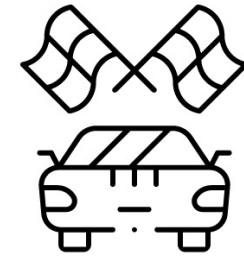
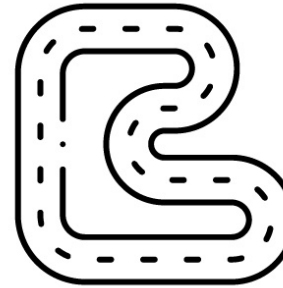
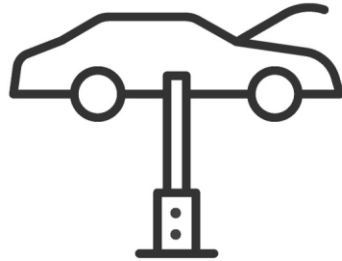
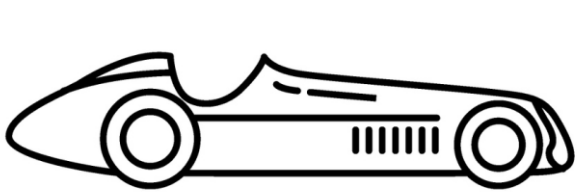
- Protect data veracity.
- Collect enough data to train and test the ML model and ingest it into the pipeline.
- Apply labels to training data with known targets.

Protecting data veracity

- Verify the integrity of the data sources.
- Protect the ingestion pipeline.
- Validate combined datasets.
- Audit the ingestion process.



Collecting enough data for training and testing



Training data: 70-80% of data

- Data that your model learns from

Test data: 20-30%

- Data that is used to validate a model

Production data

- Data that predictions are made from in your live application using the deployed model
- Feeds additional training and tuning

Key takeaways: Collecting data



- Collecting data in the ML lifecycle is similar to an extract, load, and transform (ELT) ingestion process.
- The data engineer and the data scientist ensure that they have enough of the correct data to support training and testing the model.
- During the data collection phase, you might need to add labels to the training data.

Applying labels to training data with known targets

Processing Data for ML

Labeling provides context for the ML model

- Humans add label values to training data.
- The ML model learns from the labeled data to identify patterns.

Does the photo contain a dog?

Yes



Yes



Yes



No



Example: Tabular data labeling

months_as_customer	age	umbrella_limit	vehicle_claim	Is this fraud?
328	48	0	52080	Y
228	42	5000000	3510	Y
134	29	5000000	23100	N
256	41	6000000	50720	Y
228	44	6000000	4550	N

Examples: Common types of labeling



Computer vision

Interpret objects or categories within an image.



Natural language processing

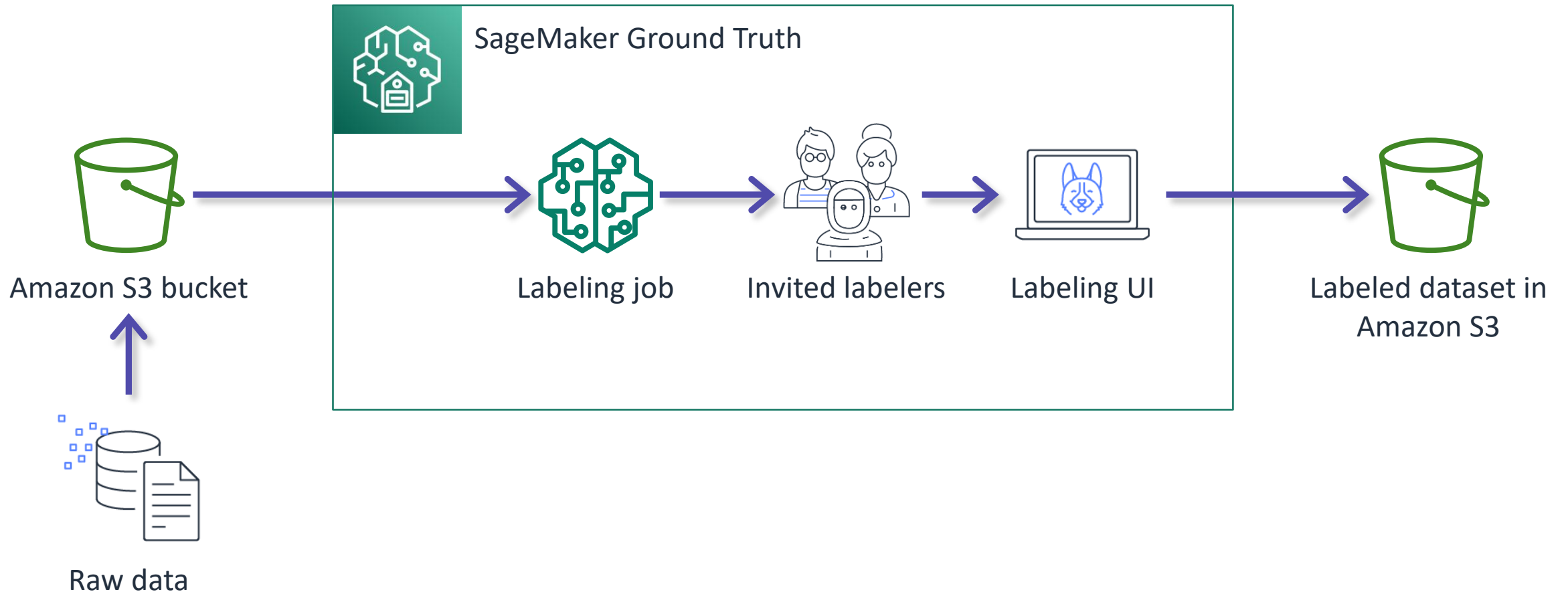
Interpret the sentiment that is expressed in text, or recognize named entities.



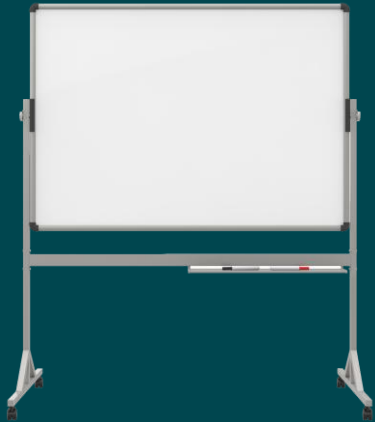
Audio processing

Interpret sounds, such as speech, wildlife, or mechanical noises.

Amazon SageMaker Ground Truth



Activity: Labeling with SageMaker Ground Truth



- Your customer has a pet supply website where pet owners can upload photos of their pets. The customer wants the website to automatically categorize new uploads as dog, cat, or other.
- You and your classmates are the labeling workforce.

Key takeaways: Applying labels to training data with known targets

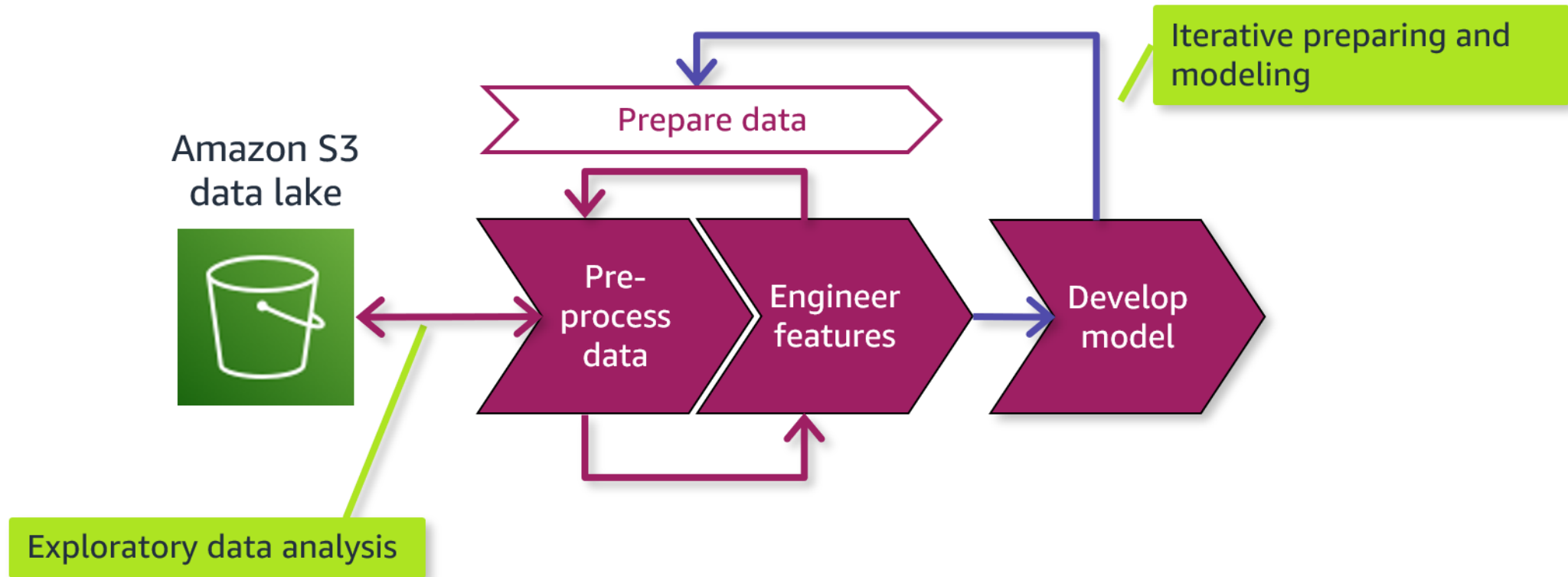


- Labeling is the process of assigning useful labels to targets in the training data.
- Common types of labeling include computer vision, natural language processing, audio processing, and tabular data.
- By using tools such as SageMaker Ground Truth, you can share labeling jobs with an expert workforce.

Preprocessing data

Processing Data for ML

The data scientist prepares the data iteratively



Preprocessing strategies

Strategy	Description	Why it is important
Clean	Remove outliers and duplicates. Fix inaccurate or missing data.	Creates a reliable dataset and improves the quality of training data.
Partition	Randomly split data into train, validate, and test sets.	Prevents ML models from overfitting. Evaluates the trained model accurately.
Scale (normalize and standardize)	Help keep target values close to normally distributed.	Makes processing easier for most ML algorithms.
Augment	Synthesize additional data from existing data (for example, generate different versions of an image).	Increases the amount of data, which can help to reduce overfitting.
Balance and unbiased	Mitigate imbalances in the presence of different feature values.	Helps to avoid inaccurate results.
Data formatting and conversion	Modify how data is represented to match requirements of the algorithm and patterns in the data.	Prepares data for precise input and output requirements of the algorithm, including the need for numeric values.

Example: Preprocessing

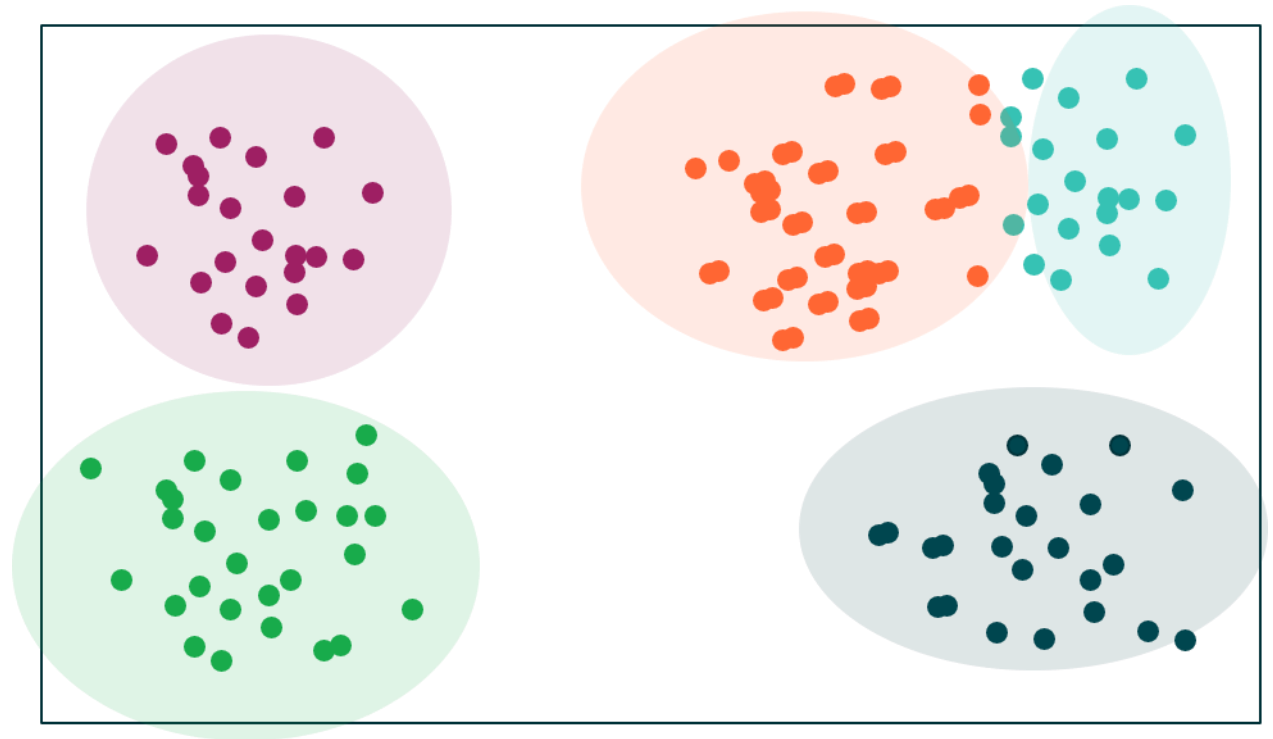
months_as_customer	age	umbrella_limit	vehicle_claim	Is this fraud?
328	48	0	52080	1
228	42	5000000	3510	1
134	29	5000000	23100	0
256	41	6000000	50720	1
228	44	6000000	4550	0

Convert to numeric field

Exploratory data analysis

- Find patterns.
- Guide preliminary selection of relevant algorithms.
- Inform feature engineering.

Example: The data scientist runs a clustering algorithm to identify customer segments that might be relevant to predicting behavior.



Key takeaways: Preprocessing data

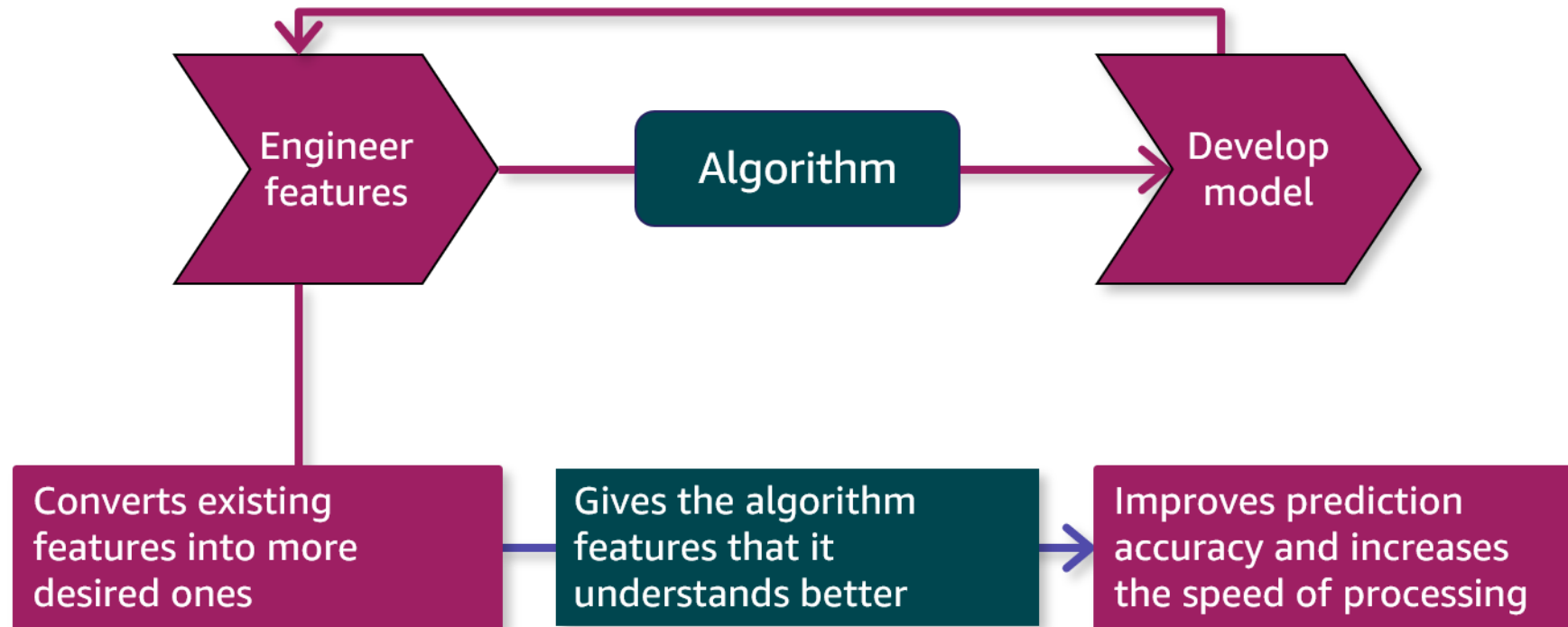


- Data preprocessing puts data into the correct shape and quality for training.
- The data scientist performs preprocessing by using a combination of techniques and expertise.
- Exploring and visualizing the data helps the data scientist get a feel for the data.
- Examples of preprocessing strategies include partitioning, balancing, and data formatting.

Feature engineering

Processing Data for ML

Feature engineering improves the usefulness of the data



Feature creation and transformation generate new features

Binning: Convert values that could fall in a continuous range to a fixed set of categories

age	Rate group 1 (25–29)	Rate group 2 (30– 44)	Rate group 3 (45– 60)
48	0	0	1
42	0	1	1
29	1	0	0
41	0	1	0
44	0	1	0



Age is converted into a set of three columns

Feature extraction and selection reduce dimensionality

Remove features that don't provide measurable value to predict the outcome

months_as_customer	age	umbrella_limit	vehicle_claim	Is this fraud?
328	48	0	52080	1
228	42	5000000	3510	1
134	29	5000000	23100	0
256	41	6000000	50720	1
228	44	6000000	4550	0

Key takeaways: Feature engineering

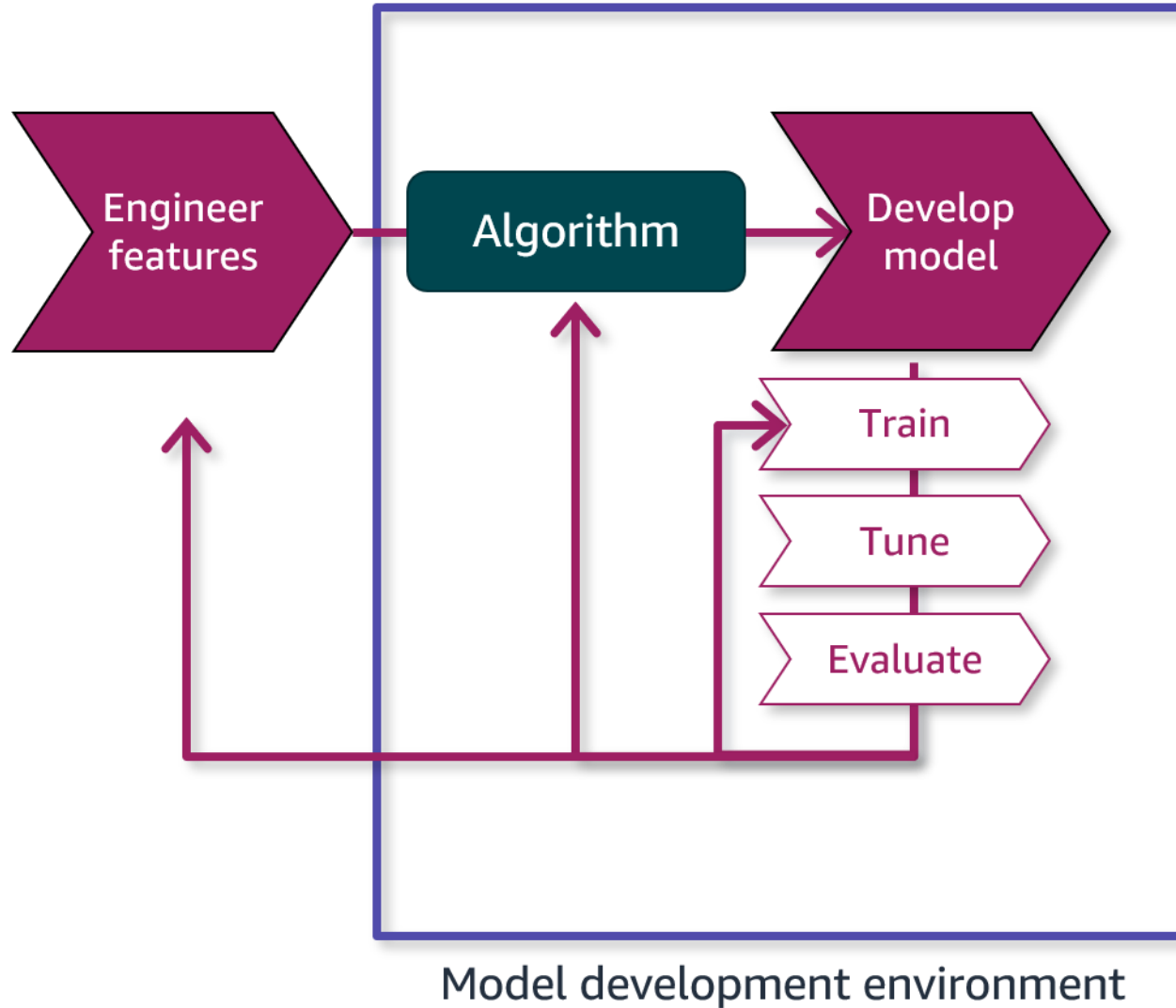


- Feature engineering is about improving the existing features to improve their usefulness in predicting outcomes.
- Feature creation and transformation focus on adding new information.
- Feature extraction and selection are about reducing dimensionality.

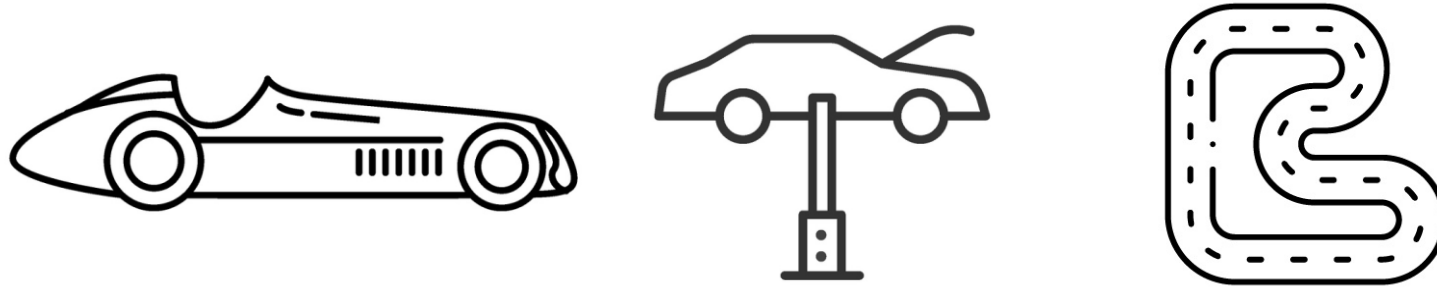
Developing a model

Processing Data for ML

Training, tuning, and evaluating a model



Training and tuning until the model meets the business goal



Train and tune

- Train the model by using the training dataset.
- Monitor error rates and tune until the desired accuracy level is reached with the training data.

Validate

- Verify results with unlabeled test data.

Types of accuracy metrics

Positive: This is a dog.

True positive: The model correctly predicted that the target was present.

False positive: The model predicted that the target was present, but it wasn't.

Negative: This is not a dog.

True negative: The model correctly predicted that the target was not present.

False negative: The model predicted that the target was not present, but it was.

- **Accuracy:** Measures the total number of correct predictions out of the total samples
- **Precision:** Measures the number of true positives out of all positive predictions
- **Recall:** Measures the number of true positives that were predicted, compared to all positives in the data
- **F1 score:** Combination of precision and recall

Scaling and cost management for model development

- Consider big data frameworks for data-intensive or loosely coupled training tasks.
- Consider high performance computing (HPC) for tightly coupled, compute-intensive training.
- Right-size the instances to optimize performance and cost for the algorithm.
- Experiment with alternative instance types.
- Start small, and scale based on results.
- Compare results from using different algorithms, features, and infrastructure.



Key takeaways: Developing a model

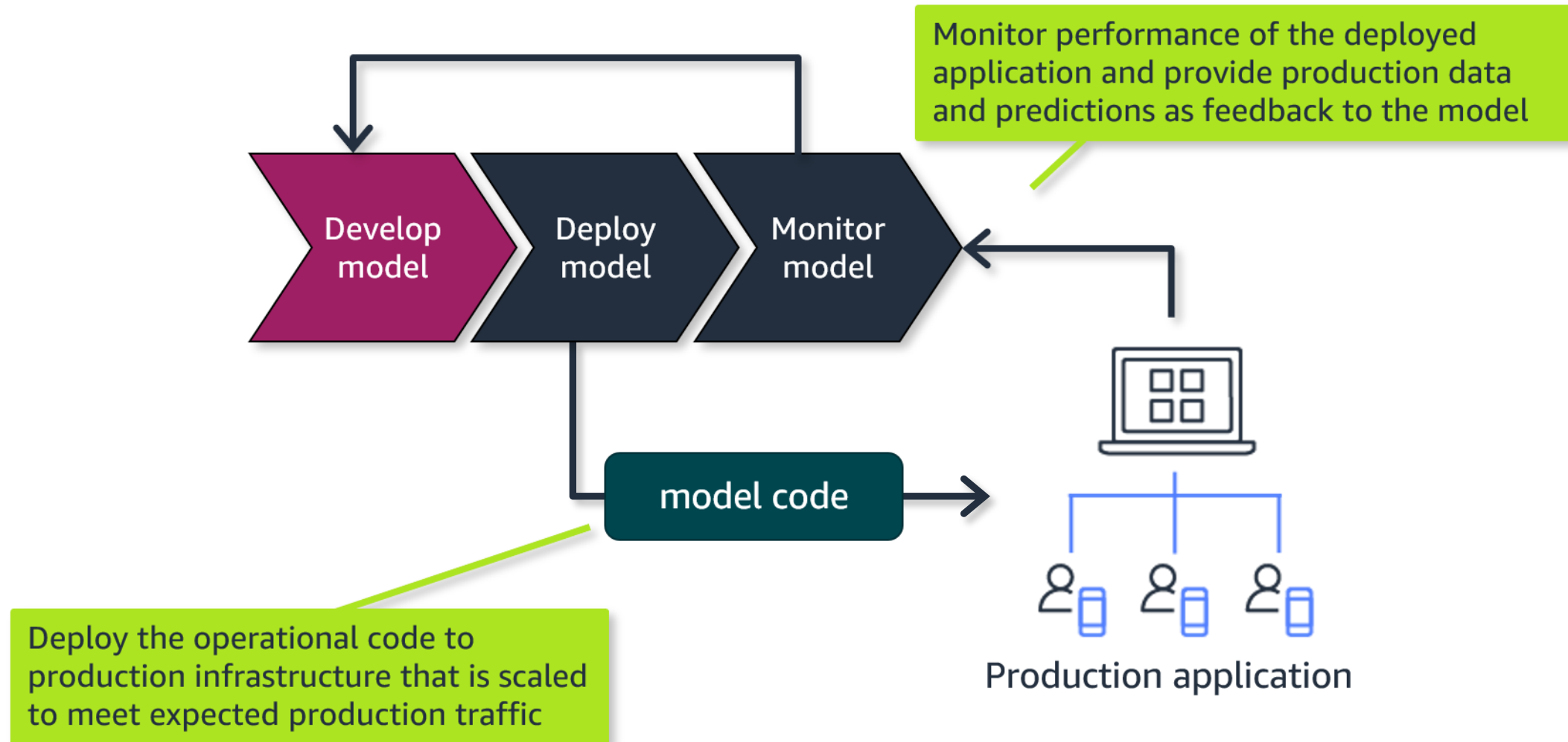


- Model development consists of model building, training, tuning, and evaluation.
- Training and tuning are iterative and continue until the accuracy rate is in line with the business goal.
- Use unlabeled test data to validate the results from the training dataset.
- Training a model is resource intensive and might require the use of big data frameworks or HPC systems.

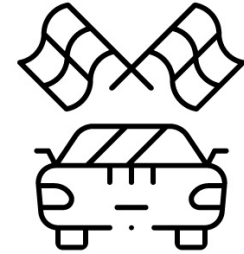
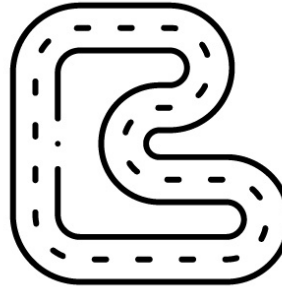
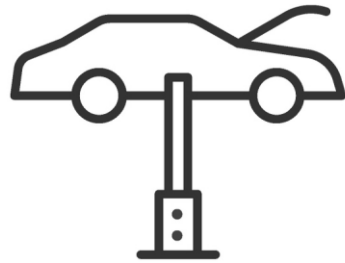
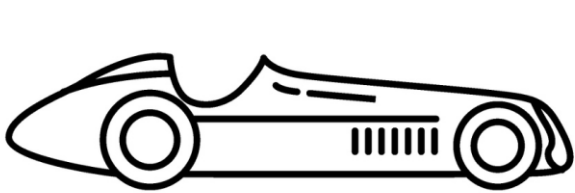
Deploying a model

Processing Data for ML

Deploying and monitoring the model in production



The production model makes inferences on new data



Continue to train, tune, and evaluate

- Production data provides a feedback loop for continual improvement of the model.
- This means ingesting new data into your model training environment.

Make inferences

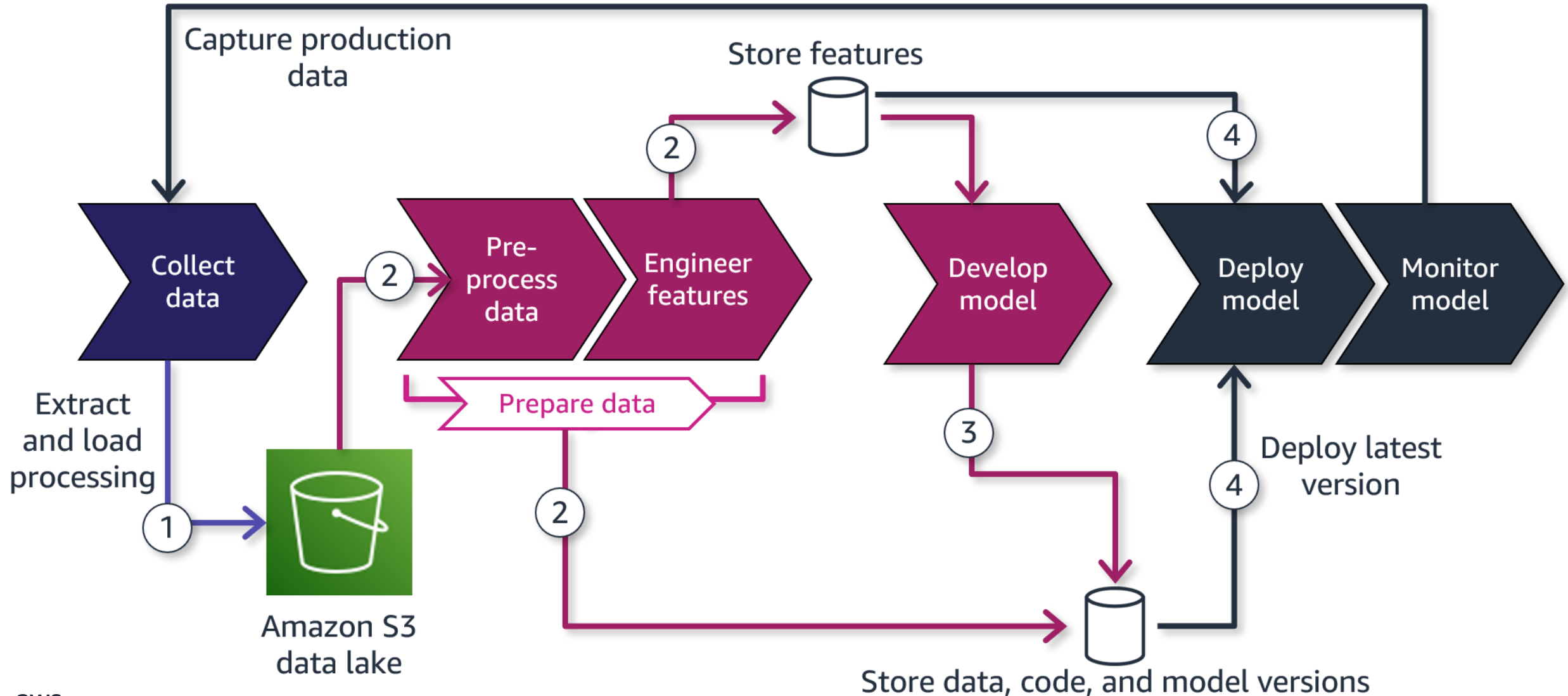
- Individual samples are inputs to the production model.
- The model provides an output for each input.

Scaling and cost management for deployed models

- Choose infrastructure that is aligned to the expected production processing.
- Use automatic scaling features to allow your application to adjust capacity to meet the need at the lowest possible cost.
- Alert on performance-related events, and proactively take action.
- Use infrastructure as code, and automate the deployment pipeline.



Adopt an MLOps approach to maintain the ML pipeline



Key takeaways: Deploying a model



- The deployment infrastructure is typically quite different from the training infrastructure.
- Inference is the process of making predictions on the production data.
- Automating the ML lifecycle is an important step in operationalizing and scaling your ML solution.
- MLOps is a deployment approach that relies on a streamlined development lifecycle to optimize resources.

ML infrastructure on AWS

Processing Data for ML

AWS ML infrastructure layers

Workflow services



SageMaker



Amazon EMR



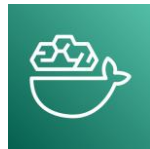
AWS Batch



Amazon EKS



DLAMI



Deep Learning Containers



AWS ParallelCluster

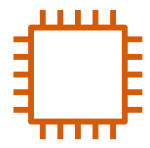


Amazon ECS

Frameworks

TensorFlow, PyTorch, MXNet, Keras, Gluon, Horovod

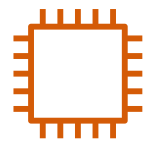
Compute, networking, and storage



EC2 P3 and P4 instances



Elastic Inference



EC2 G4 and Inf1 instances



Elastic Fabric Adapter



Outposts



Amazon S3



Amazon EFS

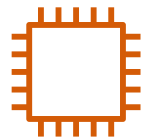


Amazon EBS



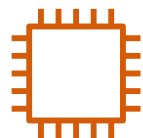
Amazon FSx

Compute and network services for ML training and deployment



EC2 P3 and P4 instances

GPU-based instances that provide high performance for ML training



EC2 G4 and Inf1 instances

GPU-based instances that are designed to provide high performance to deployed ML applications



Elastic Inference

Service that you can use to attach low-cost GPU-powered acceleration to EC2 and SageMaker instances to reduce the cost of running deep learning inference



Elastic Fabric Adapter

Network interface that provides a low-latency connection between instances, which can help large compute tasks, such as ML training, run faster

Storage services for ML training



Amazon S3

Object storage that can easily achieve thousands of transactions per second



Amazon EBS

Block storage that enables single-digit millisecond latency for high performance storage needs



Amazon EFS

File system storage that provides easy access to large ML datasets or shared code, directly from a notebook environment



Amazon FSx

File system storage for the Lustre high performance file system

Workflow services to simplify creating ML environments



DLAMI

Amazon Machine Images (AMIs) that are pre-installed with deep learning frameworks and can be used with EC2 instances



Deep Learning Containers

Docker container images that are pre-installed with deep learning frameworks



Amazon ECS

Fully managed container orchestration service that simplifies the work to deploy, manage, and scale containerized applications



Amazon EKS

Managed container service to run Kubernetes applications on AWS or on premises

Workflow services to simplify big data and HPC



Amazon EMR

Big data service that simplifies building and operating big data environments



AWS Batch

Fully managed service that can orchestrate ML training jobs on a schedule and select the optimal amount and type of resources to perform the job



AWS ParallelCluster

Open-source cluster management tool that simplifies deployment of HPC clusters

Key takeaways: ML infrastructure on AWS



- AWS ML infrastructure consists of a compute, network, and storage layer; a framework layer; and a workflow services layer.
- Services in the compute, network, and storage layer are designed to handle the compute power and speed that are required to train models and make predictions in real time.
- The workflow services layer provides tools to simplify creating and managing ML environments.

SageMaker

Processing Data for ML

SageMaker provides an integrated workbench for ML

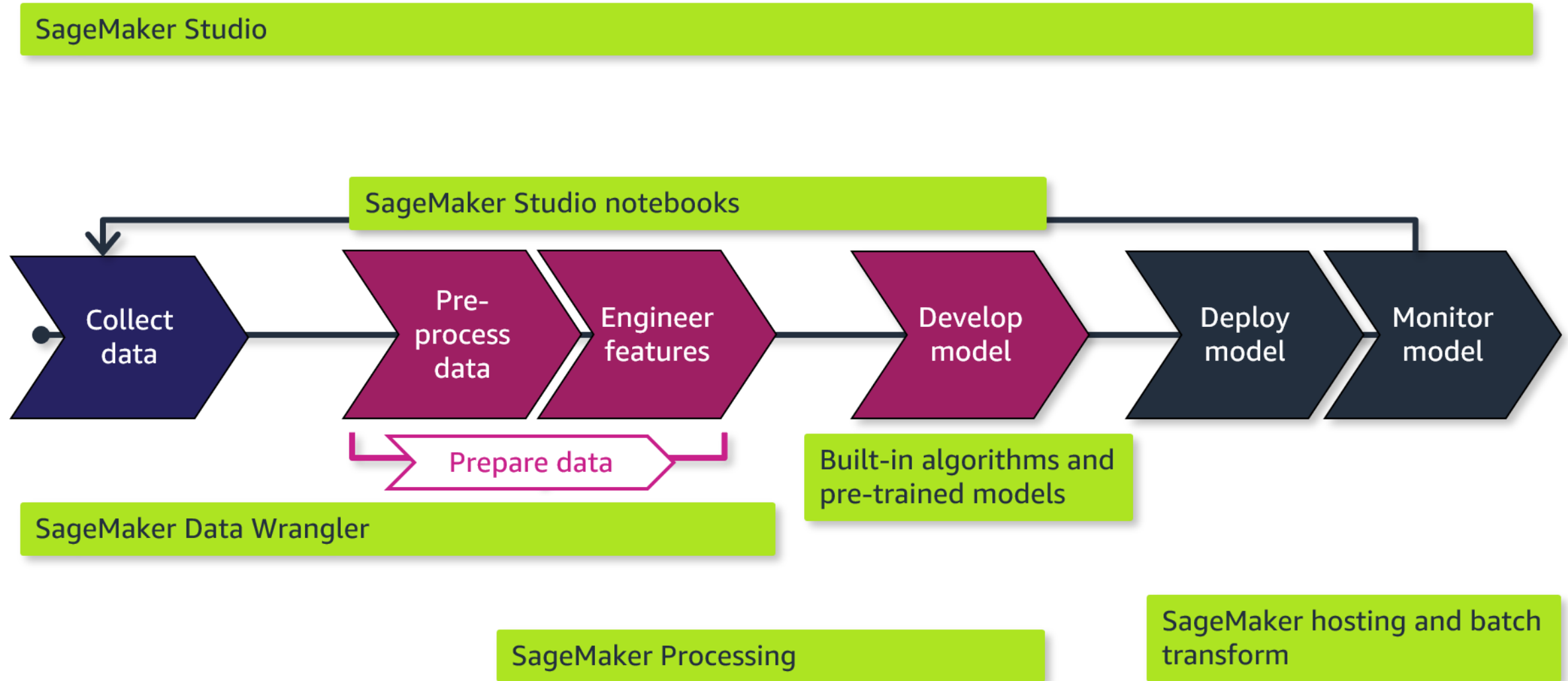
Integrated features and tools include the following:

- SageMaker Studio is a web-based visual interface where you can perform all ML development steps.
- SageMaker Data Wrangler simplifies the process of data preparation and feature engineering.
- SageMaker Processing gives you a simplified experience to run ML processing steps.
- SageMaker Canvas provides business analysts with a visual point-and-click interface that to generate accurate ML predictions on their own.

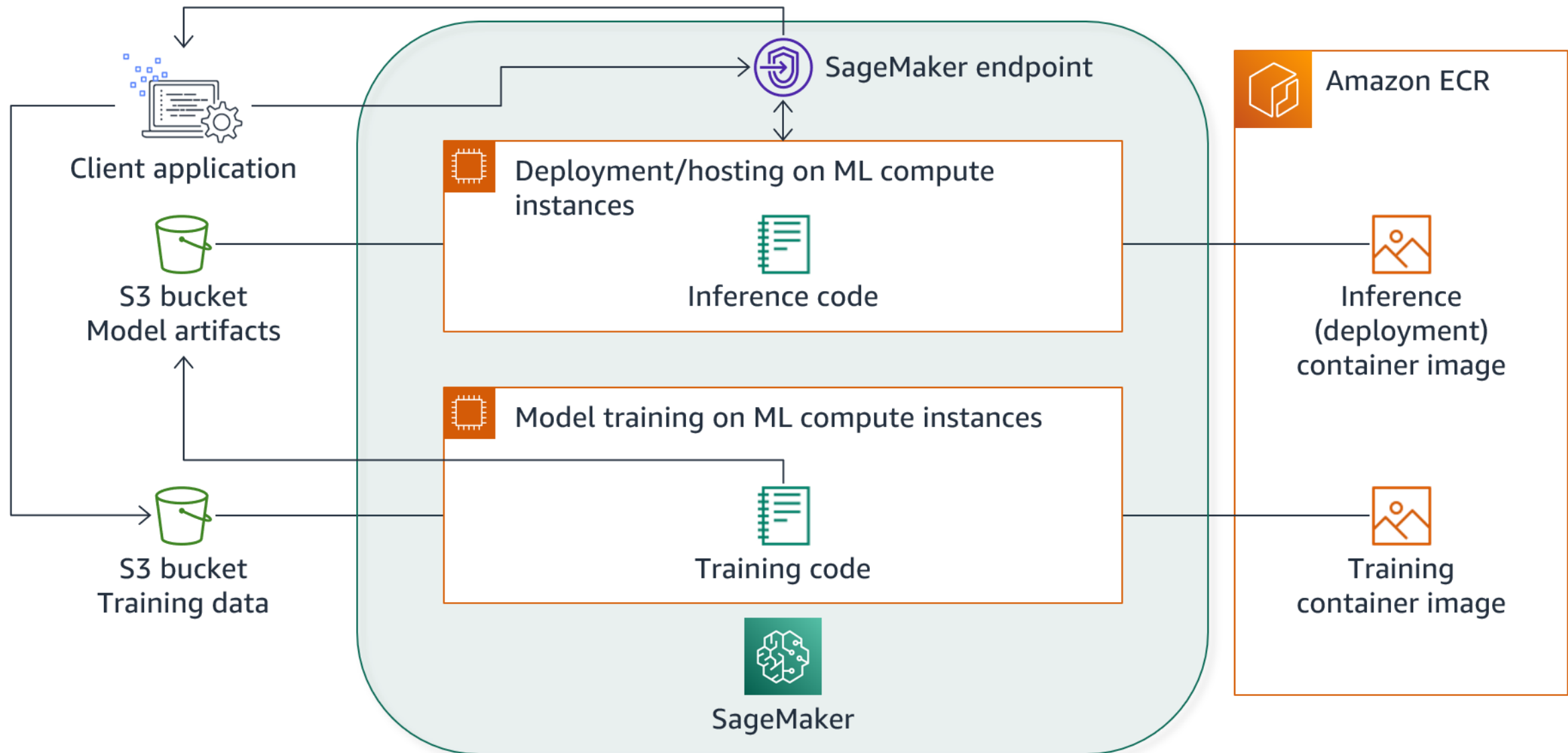


Amazon SageMaker

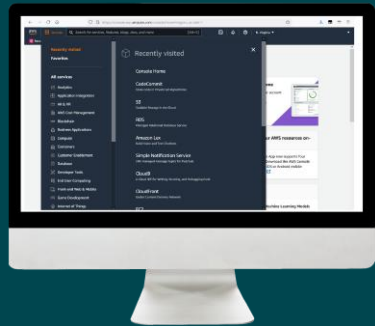
SageMaker Studio and the ML lifecycle



Training and deploying a model with SageMaker



Demo: Preparing Data and Training a Model with SageMaker



- This demo illustrates the steps to create the architecture that was presented earlier in this section and to deploy and test the fraud detection model.
- The recorded demo is included in your online course.

SageMaker Canvas generates no-code predictions

Import and manage data

- Select a file to import.
- Preview the data in the UI.
- Upload the file.

Build a model

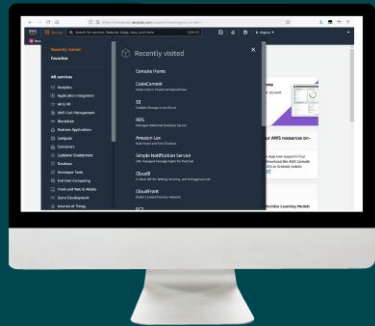
- Choose the target column from your dataset.
- Choose a recommended model.
- Preview the model.
- Choose build.

Make predictions

- Review data about the accuracy and impact of each column.
- Visualize results.
- Make predictions on new data by record or by batch.

Demo: Preparing Data and Training a Model with SageMaker Canvas

- This demo illustrates how to use SageMaker Canvas to create an ML model without the need to write or use code.
- The recorded demo is included in your online course.



Key takeaways: SageMaker



- SageMaker is a managed service that provides an integrated workbench for the ML lifecycle.
- Key components to prepare data and build models include SageMaker Studio, Data Wrangler, Studio notebooks, and Processing.
- SageMaker provides deployment options for real-time or batch inference.
- SageMaker Canvas is a no-code option that business analysts can use to build models and make predictions.

Working with generative AI

Processing Data for ML

Generative AI concepts and terminology



Foundation Model (FM):

Very large pre-trained deep learning model that can produce generalized results



Large Language Model (LLM):

A foundation model that has been trained on vast text data sets, capable of using natural language as inputs and outputs



Prompt:

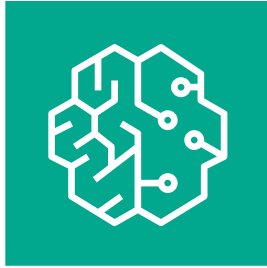
An instruction or question given to an FM or LLM as input



Prompt engineering:

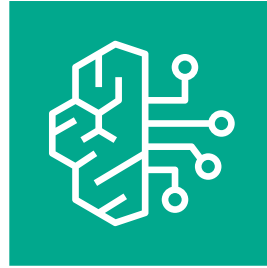
Iteratively working to develop a prompt that produces optimal results

AWS services and features for working with foundation models



Amazon SageMaker Jumpstart

- Amazon SageMaker feature for accessing leading foundation models and deploying ML solutions with fewer steps



Amazon Bedrock

- Fully managed service for connecting to leading foundation models through an API



Amazon Q Developer

- Fully managed service powered by Amazon Bedrock that provides coding assistance

Amazon Q Developer



Amazon Q Developer

- Generative AI-powered coding assistant
- Designed for developers and IT professionals
- Generates code and helps you understand, build, extend, and operate AWS applications
- Scan code for security vulnerabilities
- Secure and private by design

Amazon Q Developer supports you across the software development and ML lifecycles



Plan

- Ask questions and get referenceable and contextual guidance.
- Explain code with conversational coding.



Create

- Receive in-line code recommendations for multiple languages.
- Implement features through comments or code prompts.
- Chat in your integrated development environment (IDE) including SageMaker Studio.



Test and secure

- Generate unit tests.
- Scan project code for security vulnerabilities and get remediation suggestions.



Operate

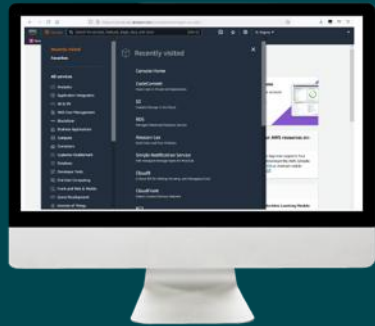
- Troubleshoot errors.
- Troubleshoot network connectivity with VPC Reachability Analyzer.



Maintain and modernize

- Modernize code with Amazon Q Developer Agent for code transformation.

Demo: Creating a Linear Regression Model Using Amazon Q Developer



- This demonstration illustrates how Amazon Q Developer can accelerate coding tasks in ML applications.
- The recorded demo is included in your online course.

Key takeaways: Working with generative AI



- Foundation models (FMs) and large language models (LLMs) are very large deep learning models that can generate content.
- Prompt engineering is the practice of writing inputs or instructions (prompts) to interact with a foundation model to generate a desired output.
- AWS offers several tools for using generative AI and interacting with foundation models. These include Amazon SageMaker Jumpstart, Amazon Bedrock, and Amazon Q Developer.
- Amazon Q Developer is a generative AI-powered coding assistant that can help you develop your applications faster and learn how to build AWS applications.

AI/ML services on AWS

Processing Data for ML

Purpose-built services for ML with less overhead



Amazon Rekognition

Detect faces, text, logos, and objects in images and video.



Amazon Textract

Extract printed text, handwriting, layout elements, and data from any document



Amazon Comprehend

Derive insights and connections from text by using natural language processing.



Amazon Transcribe

Convert speech to text.



Amazon Polly

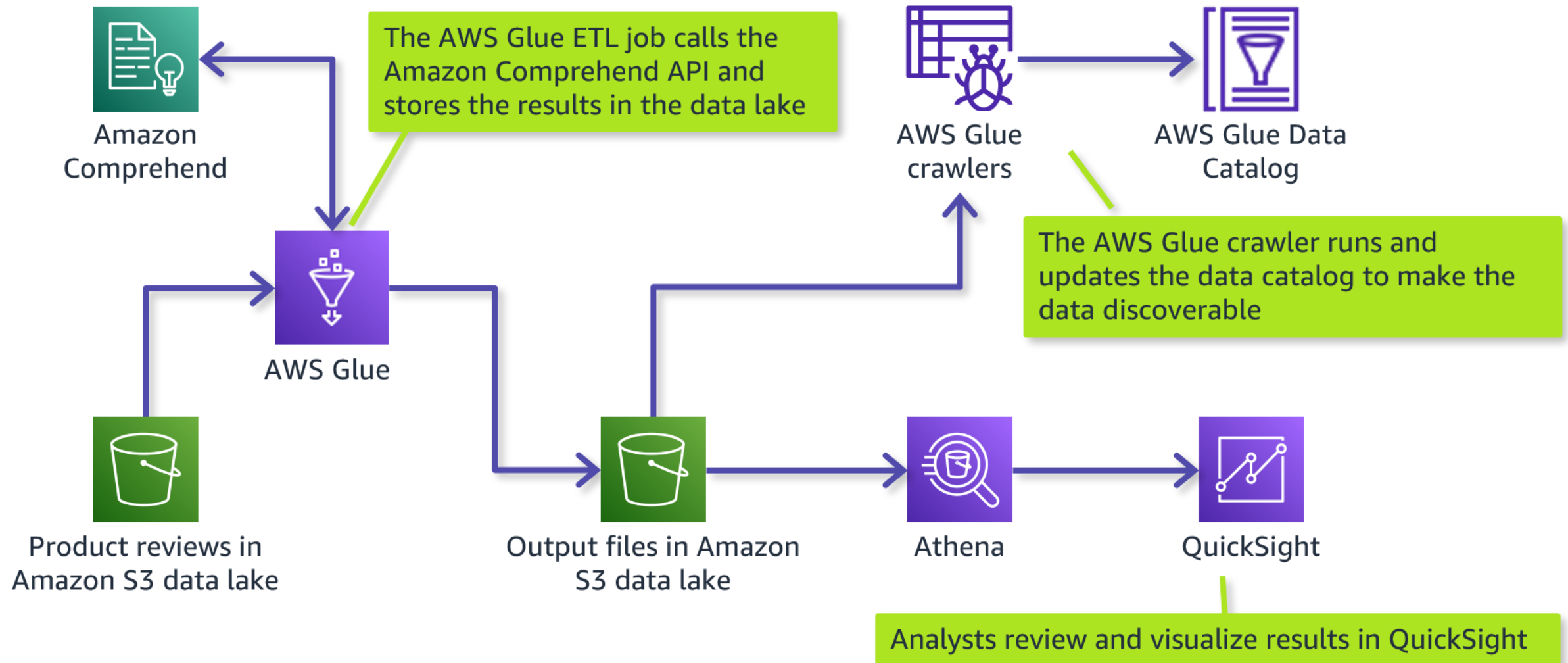
Convert text into life-like speech.



Amazon Translate

Localize content and translate large volumes of text.

AI/ML service example: Amazon Comprehend



Key takeaways: AI/ML services on AWS



- AWS offers a growing number of purpose-built AI/ML services to handle common use cases.
- Services include natural language processing, image and video recognition, and time-series data predictions.
- You can incorporate these services into your data pipelines without the burden of building custom ML solutions.

Module wrap-up

Processing Data for ML

Module summary

This module prepared you to do the following:

- Distinguish between labels, features, and samples in the context of ML.
- Describe each phase of the ML lifecycle and the roles involved in each.
- Cite questions to ask when framing an ML problem.
- Discuss key considerations and tasks during the data collection phase of the ML lifecycle.
- Give examples of preprocessing and feature engineering data transformations.
- Compare the activities and resource considerations for model development and model deployment.
- Categorize AWS ML infrastructure services based on how they might be used in the ML lifecycle.
- Describe features of Amazon SageMaker that assist with each phase of the ML lifecycle.
- Cite two AWS services that use ML for common use cases.

Module knowledge check



- The knowledge check is delivered online within your course.
- The knowledge check includes 10 questions based on material presented on the slides and in the slide notes.
- You can retake the knowledge check as many times as you like.

Sample exam question

A data scientist is working on an ML project that uses supervised learning. They just got access to a dataset with 4,000 records to use to develop their ML model. Which option reflects a best practice for working with the data?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Supervised** learning
- Dataset with **4,000 records**
- **Develop** their ML **model**
- **Best practice**

Sample exam question: Response choices

A data scientist is working on an ML project that uses **supervised** learning. They just got access to a dataset with **4,000 records** to use to **develop** their ML **model**. Which option reflects a **best practice** for working with the data?

Choice	Response
A	Remove labels from the dataset so that the model learns on its own.
B	Split the dataset into a training dataset of 3,200 records and a test set of 800 records.
C	Split the dataset into a training dataset of 2,000 records and a test set of 2,000 records.
D	Use as many records as needed to reach the desired accuracy, and remove labels from the remaining data for testing.

Sample exam question: Answer

The correct answer is B.

Choice	Response
--------	----------

B	Split the dataset into a training dataset of 3,200 records and a test set of 800 records.
---	---

Thank you



Corrections, feedback, or other questions?

Contact us at <https://support.aws.amazon.com/#/contacts/aws-academy>.

All trademarks are the property of their owners.